Context Engineering & Reliability by Michaël Bettan

Definition

Context engineering is the per-call discipline of choosing which tokens enter a model's finite context window (instructions, tool definitions, exact facts, retrieved memories, history and tool results), in what form and in what order, and what gets persisted outside the window so an agent stays correct across long sessions, multiple agents and partial failures.

What context engineering solves for

A model is stateless: at any step it knows only what the harness placed in the window for that call. Long sessions overflow the window, summaries blur exact numbers, verbose tool output crowds out instructions, subagents inherit noise or receive too little, and failed lookups quietly become "no data". Reliability is the other half of the same problem: errors must travel with enough structure to act on, unresolvable uncertainty must reach a human, and every synthesized claim must keep its source.

> **MENTAL MODEL**
> the context window is a desk, not a filing cabinet. The model works only with what is on the desk this turn. Context engineering decides what goes on the desk, in which pile and in which order, and what gets filed away (state, files, memory) with a label that lets the agent fetch it back.



- **Scope of this guide:** context budget and placement, long sessions, sessions and memory, multi-agent handoffs, then the reliability layer (error propagation, validation loops, escalation, human review, provenance), caching and security.
- **Vendors covered:** Claude (Messages API, Agent SDK, Claude Code) and Google (ADK, Agent Platform Sessions and Memory Bank, Gemini API). Mechanisms differ; the decisions are the same.
- **Related guides:** Claude Agent SDK for session operations and MCP `isError`, ADK for state prefixes and memory services, Agentic design patterns for the memory taxonomy, Agent Platform for managed Sessions and Memory Bank, Claude Code for `CLAUDE.md` and `/compact`, AgentOps for evaluation.
- **Official documentation:** Claude context windows, context editing, compaction, memory tool and prompt caching; Claude Code memory and Agent SDK sessions; ADK state, memory, compaction and caching; Memory Bank; Gemini API caching.
- **Further reading:** the agent-as-context-curator framing is credited to *Introduction to Agents* (Blount, Gulli, Saboo, Zimmermann and Vuskovic; Google, 2026). The attention-budget view is credited to Anthropic's engineering post Effective context engineering for AI agents. The position effect is from Liu et al., Lost in the middle: how language models use long contexts (TACL 2024).

Prompt vs. Context vs. Memory

Layer/approach	Type	Role & behavior
Prompt engineering	Authoring	Wording, structure and examples of one instruction; written once, reused on every call
Context engineering	Per-turn assembly	Selects, orders, compresses and isolates everything in the window on every call, including tool results and history
State	Exact working data	Key/value values the harness reads and writes deterministically (IDs, flags, step outputs); never approximated
Memory	Long-term recall	Facts extracted across sessions and fetched by search; recall is probabilistic and can be stale
Retrieval (RAG)	Shared knowledge	Documents valid for every user (policies, manuals); not a user's personal history
Reliability engineering	Failure handling	Structured errors, validation, escalation rules, calibrated review and provenance, so failures stay visible and actionable

Context levers

- **Pin** keeps decision-critical values verbatim in a block that is re-sent every turn and never summarized (amounts, dates, identifiers, limits, promises made to the user).
- **Prune** removes what the model no longer needs: old tool results, irrelevant fields, superseded drafts, finished subtasks.
- **Partition** gives a noisy subtask its own window (a subagent or an agent-as-tool) and returns only a distilled, structured result.
- **Persist** moves information out of the window into state, files or memory, together with a way to fetch it back (a key, a path, a search tool).
- **Place** orders what remains: stable content first so it caches, key facts at the edges of long inputs, the question last.
- **Mapping to the common framing:** pin and persist correspond to *write*, retrieval to *select*, prune to *compress* and partition to *isolate* (the four strategies in the Agentic design patterns guide); place adds ordering, which matters for both recall and caching.

KEY RULE

every token competes for the same attention budget. A larger window raises the ceiling but does not remove the competition: adding irrelevant tokens lowers recall of the relevant ones.

Per-turn assembly workflow

1. **Load:** fetch session events, exact state and the pinned facts block for this user and task.
2. **Retrieve:** add memories or documents relevant to the current input, or expose a search tool and let the agent decide.
3. **Prune:** clear stale tool results, project tool output down to the needed fields and compact old turns once a token threshold is crossed.
4. **Place:** stable prefix first, then pinned facts and retrieved items, then recent history, then the current input.
5. **Call:** count tokens before sending when close to the limit; reserve room for the output.
6. **Validate:** check tool results and model output against schemas and business rules; route each error by category.
7. **Persist:** commit state changes, update pinned facts, write the scratchpad or manifest and extract memories when the session ends.

When not to engineer context

- **Short, single-turn tasks:** one well-written prompt with the needed document is enough; pinning, compaction and memory add cost with no benefit.
- **Everything already fits and stays relevant:** a 20-turn chat with small tool results needs no compaction; measure before adding machinery.
- **Shared reference knowledge:** put it in retrieval or the stable prefix, not in per-user memory.

Context budget and placement

Budget arithmetic

- **The window holds input and output:** system prompt, tool definitions, history, tool results and the reply all count; reserve output headroom (`max_tokens` on Claude, `max_output_tokens` on Gemini) before filling the rest.
- **Window sizes differ by model:** current Claude models default to a 1M-token window with up to 128k output tokens, while legacy models keep 200k; check the model page rather than assuming.
- **Count, don't guess:** Claude exposes a token counting endpoint (`POST /v1/messages/count_tokens`, returning `input_tokens`) and Gemini exposes `count_tokens`; character heuristics (about four characters per token in English) are only rough planning numbers.
- **Recount when you switch models:** tokenizers change between model generations (Anthropic documents roughly 30% more tokens for the same text with the tokenizer introduced in Opus 4.7), so a prompt that fit before can overflow or cost more after an upgrade.
- **Let the agent see its budget:** recent Claude models receive a context-awareness note after tool calls with tokens used and remaining, which helps them wrap up or save notes before the limit.
- **Tool definitions are a fixed tax:** every tool schema is sent on every call. Thirty tools with verbose descriptions consume budget before the first user word; scope tools per agent.
- **Context rot:** recall of a specific detail degrades as total tokens grow, well before the hard limit. Treat the usable window as smaller than the advertised one.

Placement

- **Lost in the middle:** models use information at the start and end of a long input more reliably than information buried in the middle (Liu et al.). Put the facts that drive the decision at the top or bottom, never between two long documents.
- **Long documents first, question last:** for large inputs, place documents above the instructions and the question at the end; Anthropic's long-context guidance reports better answers with this order.
- **Label what you place:** wrap documents and facts in named sections or XML-style tags (`<case_facts>`, `<document source="..." date="...">`) so the model and the reader can refer to them precisely.
- **Restate the critical constraint near the end:** for very long contexts, a one-line reminder of the hard limit right before the question costs little and counters drift.

[!NOTE] *Directional figures:* the "about four characters per token" heuristic and vendor-reported quality gains from document placement are rules of thumb. Measure on your own prompts with the token counting APIs and an evaluation set.

Pinned facts block

Scenario: outage planning at Tideline Utilities

A grid engineer and a planning agent spend three hours scheduling a feeder outage.

- **What went wrong:** after the harness summarized older turns, the agent proposed a switching plan with "roughly 4 MW" of transferable load (the engineer had said 4.2 MW) and dropped the dialysis clinic that must stay energized.
- **Root cause:** the exact values lived only in conversation text, and summarization turns exact values into approximations and drops "minor" exceptions.
- **Fix:** a pinned block, owned by the harness and re-injected at the top of every call, that summarization never touches.

```
<case_facts updated="2026-11-03T10:42Z">
  feeder: F-2207 | outage window: 2026-11-14 06:00–09:30 local
  transferable load cap: 4.2 MW (engineer, turn 6)
  must stay energized: Harbor Dialysis Center (meter 88-1043)
  crew limit: 6 | switching order requires supervisor sign-off
</case_facts>
```

- **Update rule:** the harness (or a dedicated tool such as `update_case_fact(key, value, source)`) changes the block; the model never rewrites it freehand.
- **Provenance inside the block:** record who stated each value and when, so a later contradiction can be resolved instead of silently overwritten.
- **Anti-pattern (summary as the only record):** trusting a running summary to carry amounts, dates and identifiers. Summaries are for narrative; exact values belong in pinned facts or state.

Trimming tool output at the boundary

Scenario: fleet telemetry

A dispatch agent calls `get_vehicle_status`, which returns 60 fields for each of 200 trucks.

- **Symptom:** after three calls the window is mostly raw JSON; the agent forgets the dispatcher's delivery-priority rule stated at the start.
- **Fix:** project at the tool boundary: return only `vehicle_id`, `location`, `fuel_pct` and `eta_next_stop`, filtered to the region in question. Aggregate when the agent needs a count, not rows.
- **Anti-pattern (ask the model to ignore it):** "focus only on fuel level" in the prompt still pays the tokens and still dilutes attention. Remove data in code, before it reaches the window.

Technique	Where	Effect
Field projection	Tool implementation	Return the 4 fields needed, not 60
Server-side filtering and pagination	Tool parameters (`region`, `limit`, `cursor`)	The agent asks for the slice it needs
Aggregation	Tool implementation	"37 trucks below 20% fuel" instead of 200 rows
Post-tool hook or callback	Claude `PostToolUse` hook; ADK `after_tool_callback`	Normalize or truncate results the tool owner cannot change
Reference instead of payload	Files API, ADK artifacts	Store the full result; pass a handle and a summary

Long sessions and compaction

Symptoms of an overloaded session

- **Generic answers:** the agent refers to "typical patterns" instead of the specific files, IDs or decisions found earlier.
- **Constraint drift:** rules stated early stop being applied.
- **Repetition:** the agent re-runs searches whose results it already has, because those results are buried or were cleared without a note.

Techniques by vendor

Technique	Claude	Google ADK and Gemini	Use when
Summarize old turns (compaction)	Messages API compaction (on demand or at a token threshold); Claude Code auto-compact and `/compact [focus]`	`EventsCompactionConfig` on the `App` (sliding window or token threshold)	The transcript keeps growing and older detail matters only as narrative
Clear stale tool results (context editing)	`clear_tool_uses_20250919` edit; `clear_thinking_20251015` for old thinking blocks	Prune or rebuild history in a callback from state	Tool payloads dominate and are re-fetchable
Start fresh with a brief	New session seeded with a structured summary; `/clear` in Claude Code (the old session stays reachable via `/resume`)	New session; state and memory carry what matters	Many prior results are stale (files changed, data updated)
Externalize to files	Scratchpad file, memory tool directory (`/memories`), `CLAUDE.md`	Artifacts, state, Memory Bank	Findings must survive compaction, restarts or handoffs
Isolate exploration	Subagents (fresh window, summary back)	`AgentTool`, `include_contents='none'`	Discovery generates bulky output the parent does not need

Claude and ADK knobs

Mechanism	Identifier and key parameters	Behavior
On-demand compaction	Top-level `compaction: {"type": "summarize", "instructions": ...}` (beta `compact-2026-09-04`)	Your app decides when; Claude returns a signed `compaction` block that you send first in later `messages`, in place of the summarized turns
Threshold compaction	Edit `compact_20260112` (beta `compact-2026-01-12`); `trigger` defaults to 150,000 input tokens; optional `instructions`; `pause_after_compaction`	Server summarizes when input crosses the trigger; pausing returns `stop_reason: "compaction"` so you can inspect or re-pin facts
Tool-result clearing	Edit `clear_tool_uses_20250919` (beta `context-management-2025-06-27`); `trigger`, `keep` (recent tool uses kept), `clear_at_least`, `exclude_tools`, `clear_tool_inputs`	Replaces old tool results with placeholders once the trigger is crossed
Thinking clearing	Edit `clear_thinking_20251015` with `keep: {"type": "thinking_turns", ...}`; listed first in `edits`	Drops older thinking blocks
Claude Code	`/compact [focus]`, auto-compact (`autoCompactWindow` setting), `/context`, `/clear`	Project-root `CLAUDE.md` is re-read from disk after compaction, so durable rules belong there
Agent SDK	`compact_boundary` system message with `compact_metadata` (`pre_tokens`, `trigger`)	Lets the harness log compactions and re-inject pinned facts afterwards
ADK	`EventsCompactionConfig(compaction_interval, overlap_size)` or token mode (`token_threshold` with `event_retention_size`, set together) on `App`	Summarizes older event ranges into a compacted event; recent events stay verbatim

- **Compaction instructions matter:** tell the summarizer what to keep verbatim (open decisions, file paths, IDs, user commitments). Both Claude compaction modes accept `instructions`, `/compact` accepts a focus, and a `CLAUDE.md` section can describe what compaction must preserve.
- **Compaction is lossy by design:** keep pinned facts and state outside the summarized region so compaction can never alter them.
- **Clearing needs a breadcrumb:** when a tool result is cleared, leave a short placeholder (what was fetched and how to fetch it again) so the agent does not assume the data never existed.

Scenario: migrating 1,400 cron scripts at Quarry Data

An agent is moving a decade of cron scripts to a workflow orchestrator across a large monorepo, over several days.

1. **Discovery in subagents:** "list every script that writes to the billing schema" runs in subagents; each returns a table of paths, schedules and owners rather than raw grep output.
2. **Scratchpad file:** the coordinator records decisions in `MIGRATION_NOTES.md` (naming convention chosen, scripts deferred and why) and re-reads it before each phase.
3. **Phase summaries:** at the end of each phase the coordinator writes a summary of verified findings and injects it into the briefs for the next phase's subagents.
4. **Crash-recovery manifest:** every agent writes its progress to a known path (`.agent-state/manifest.json`: phase, completed items, pending items, last artifact); after a crash or a new session, the coordinator loads the manifest and resumes instead of rediscovering.
5. **Resume or restart:** if the repo changed overnight, the coordinator either resumes and names the changed files for targeted re-analysis, or starts a fresh session from the manifest when most earlier results are stale.

> **COMMON TRAP**
> compacting a session whose tool results are stale. Compaction preserves the stale conclusions in shorter form. Re-verify or start fresh from a summary of the findings that are still valid.

Session vs. State vs. Memory

Store	Holds	Lifetime	Recall
Session (history)	The ordered turns and tool calls of one conversation	One conversation (resumable)	Replayed into the window, usually compacted
State	Exact key/value working data	Session, user or app scope	Deterministic read by key
Memory	Extracted facts about a user or domain, across sessions	Long term, with consolidation and expiry	Search (similarity or tool call); probabilistic
Files and artifacts	Large or binary outputs, scratchpads, manifests	Until deleted	Read by name or path

Cross-vendor map

Concern	Claude	Google ADK and Agent Platform
Statelessness	The Messages API keeps nothing between calls; the client resends history every time	The `Runner` loads session events from a `SessionService` before each model call
Session handle	Agent SDK and Claude Code session IDs: continue, resume by ID or name, fork into a new ID	`Session` keyed by `app_name`, `user_id`, `session_id`
Session storage	Local session transcripts (Agent SDK, Claude Code); your own database for raw API apps	`InMemorySessionService` (dev), `DatabaseSessionService`, `VertexAiSessionService` (Agent Platform Sessions)
Exact state	Your application store, or files the agent reads	`session.state` with prefixes: none (session), `user:`, `app:`, `temp:` (never persisted)
Long-term memory	Memory tool (`memory_20250818`): client-side files under `/memories` that Claude views, creates, edits, renames and deletes; Claude Code `CLAUDE.md` hierarchy plus auto memory (`MEMORY.md`)	`MemoryService`: `InMemoryMemoryService` (dev, keyword match), `VertexAiMemoryBankService` (Memory Bank: extraction and consolidation), `VertexAiRagMemoryService`
Memory read path	The agent calls the memory tool when it decides to	`load_memory` (agent decides) or `PreloadMemoryTool` (every turn)
Large outputs	Files API, local files	`ArtifactService` (versioned artifacts)

Where does it belong?

Information	Store	Why
Ticket ID, approved budget, chosen option	State or pinned facts	Must be exact; read by key every turn
"Prefers reports in French"	User-scoped state or long-term memory	Survives sessions; small and stable

Information	Store	Why
Facts learned over many conversations	Long-term memory with consolidation	Needs dedup and updates when facts change
A 40-page contract	File or artifact plus retrieval	Too large to replay; cite sections on demand
Progress of a multi-day task	Manifest or scratchpad file	Survives crashes and new sessions
Value needed only within this turn	`temp:` state or a local variable	Must not leak into the next turn
The tone of yesterday's chat	Nothing	No future decision depends on it

Session operations

- **Continue** picks up the most recent session; **resume** reopens a specific session with its full history; **fork** copies the history into a new session ID and leaves the original untouched (see Claude Agent SDK sessions).
- **Fork for divergent exploration:** compare two approaches from the same analysis baseline. A fork copies the transcript, not the filesystem; isolate file edits (worktrees or checkpoints) when forks change code.
- **Resume when the context is still valid:** if a few files changed, resume and name them. If most tool results are stale, start a new session seeded with a structured summary of what is still true.

Memory write policy

- **Extract at boundaries:** generate memories at session end or at task milestones (ADK: `add_session_to_memory` on a Memory Bank service, which calls `GenerateMemories` in the background), not on every turn.
- **Consolidate, don't append:** a new fact that contradicts an old one updates it ("moved to Lyon" replaces "lives in Paris"); Memory Bank consolidates extracted facts against existing memories in the same scope, while append-only stores accumulate contradictions.
- **Direct writes for explicit facts:** when the user states a fact to remember, write it directly (Memory Bank `CreateMemory`, or the Claude memory tool's `create` command) instead of waiting for extraction.
- **Scope every record:** key memories by user (and tenant); Memory Bank `RetrieveMemories` matches the exact scope before any similarity search, and your own stores must do the same.
- **Expire what goes stale:** set a time-to-live for facts with a shelf life (Memory Bank supports TTL per instance or per memory and keeps revisions); review the rest periodically.
- **Assume the window resets:** with the Claude memory tool enabled, Claude is instructed to check its memory directory before starting and to record progress as it works, because any context can be compacted or cleared.
- **Anti-pattern (memory as a transcript dump):** saving raw transcripts as "memory" makes retrieval noisy and copies sensitive data into a long-lived store.

> **COMMON TRAP**
> in ADK, `user:` and `app:` state looks persistent but disappears on restart if `InMemorySessionService` is still configured. Durable scopes need `DatabaseSessionService` or `VertexAiSessionService`.

Multi-agent context isolation and handoffs

- **Isolation is the point:** a subagent starts with a fresh window. On Claude, an Agent SDK subagent receives only its own system prompt and the task prompt the coordinator writes; it does not see the parent's conversation. In ADK, `AgentTool` runs the child in an isolated session, while `sub_agents` transfer hands over the shared conversation.
- **Brief, don't forward:** pass a curated brief, not the transcript. Forwarding everything defeats isolation and imports the parent's noise.
- **Return structure, not prose:** the subagent returns findings in a schema the coordinator can merge, with the metadata needed downstream (source, date, location, coverage gaps).

Brief template

```
GOAL:         Identify the ultimate beneficial owners of Arvalis
Components GmbH.
CONSTRAINTS:  Public registries and filings only; stop after 12
tool calls.
KNOWN FACTS:  Registry ID HRB 73319 (Munich); parent named in 2024
filing: Velden Holding.
OUT OF SCOPE: Sanctions screening (handled by another agent).
RETURN:       JSON list of {owner, share_pct, source_url,
document_date, excerpt},
              plus "gaps": sources you could not reach and why.
```

Scenario: supplier due diligence at Kestrel Procurement

A coordinator assesses a new supplier with three subagents: registry, news and sanctions.

- **Registry subagent:** returns ownership claims with filing dates and excerpts.
- **News subagent:** returns dated articles; flags that one article reports a 2026 ownership change.
- **Sanctions subagent:** returns match results per name, or an explicit error per list it could not query.
- **Coordinator:** merges claims keyed by source, keeps conflicting values side by side with dates, and lists coverage gaps in the final report.

Handoff mistake	Consequence	Fix
Brief says "check the supplier" with no identifiers	Subagent searches the wrong company with a similar name	Include exact identifiers and known facts
Subagent returns a paragraph	Coordinator cannot merge or cite	Require a schema with source fields
Parent transcript forwarded	Subagent window filled with irrelevant history	Curated brief only
No coverage field	Missing sources look like "nothing found"	Require a `gaps` list
Coordinator re-summarizes subagent output loosely	Attribution lost	Merge structured claims; never paraphrase away sources

Error propagation

Access failure vs. valid empty result

Scenario: the sanctions list that timed out

The sanctions subagent queries two watch lists. One returns no match; the other times out. The wrapper turns the timeout into an empty list.

- **Outcome:** the coordinator reports "no sanctions matches on either list" and the supplier is approved. The second list was never checked.
- **Rule:** an empty result is a successful answer ("I looked and found nothing"). A failure is the absence of an answer ("I could not look"). They must never share a representation.

```
{
  "status": "error",
  "error_category": "transient",
  "retryable": true,
  "source": "watchlist_b",
  "attempted": "screen(name='Velden Holding',
list='watchlist_b')",
  "attempts": 2,
  "partial_results": [{"list": "watchlist_a", "matches": []}],
  "suggested_next_step": "retry in 10 minutes or mark watchlist_b
unverified in the report"
}
```

Category	Example	Retry?	Caller should
Transient	Timeout, rate limit, 503	Yes, bounded with backoff	Retry locally first, then propagate with partial results
Validation	Malformed date, unknown field	After fixing the input	Correct the call or ask the user for the missing value
Permission	Token lacks scope	No	Stop that path; escalate or use an authorized route
Business rule	Order exceeds approval limit	No	Explain the rule and offer the allowed alternative
Not found	ID does not exist	No	Treat as a real answer; confirm the identifier if it came from the user

Recover locally, propagate what you cannot fix

1. **Retry transient failures inside the tool or subagent** with a small bound and backoff.
2. **Fall back to an equivalent source** if one exists and record that you did.
3. **Return partial results** with an explicit failure record for the part that failed.
4. **Let the coordinator decide** whether to retry later, proceed with an annotated gap or escalate.

- **Mechanisms:** on Claude, a tool result can carry `is_error: true` (MCP: `isError`), and the loop continues so Claude can react; put the structured payload in the content. In ADK, tools conventionally return a dict with a status and an error message; `ReflectAndRetry` `ToolPlugin` retries raised exceptions by default.
- **Anti-pattern (silent suppression):** returning `[]`, `null` or "success" on failure. Downstream agents read it as a fact.
- **Anti-pattern (abort everything):** killing the whole workflow because one source failed throws away valid partial work. Continue, annotate the gap.
- **Anti-pattern (generic error):** "operation failed" tells the caller nothing about whether a retry, a different input or a human is needed.
- **Anti-pattern (unbounded retry):** a subagent retrying forever burns budget and hides an outage from the coordinator.

Validation and retry loops

- **Separate syntax from meaning:** structured outputs (Claude strict tool use or JSON outputs; ADK `output_schema` and Gemini `response_schema`) guarantee valid shape. They do not guarantee the values are right.
- **Validate in code:** schema checks, cross-field rules (end date after start date, units consistent) and recomputation of anything derivable. Do not ask the model whether its own output is correct as the only check.
- **Retry with the specific error:** feed back the exact failing rule and the offending value (" `calibration_date` 2026-13-02 is not a valid date; month must be 01–12"). A bare "try again" wastes the attempt.
- **Bound the loop:** two or three retries, then route to human review with the last error attached.

Scenario: calibration certificates at Calloway Metrology

An extraction agent reads calibration certificates: instrument serial, calibration date, twelve measurement points with tolerances and a pass or fail verdict.

- **Recompute, then compare:** the validator recomputes the verdict from the points and tolerances. If the certificate says "pass" but two points are out of tolerance, the record is flagged as a discrepancy rather than "corrected" by the model.
- **Retry helps when the value is present:** a misread date or a value in the wrong field is fixable with targeted feedback.
- **Retry does not help when the value is absent:** if a certificate has no next-due date, more retries produce invented dates. Make such fields nullable, require a reason (`"not_stated_in_document"`) and accept null as a valid answer.

Failure	Retry helps?	Better response
Malformed JSON or missing required key	Rarely needed	Use structured outputs so it cannot happen
Value in the wrong field or wrong format	Yes	Retry with the exact rule that failed
Value not present in the source	No	Nullable field plus reason; never force a value
Source contradicts itself	No	Extract both values and flag for review
Model keeps failing the same rule	No, after the bound	Human review with the last error attached

> **COMMON TRAP**
> making every field required "to improve completeness". Required fields with no source value invite fabrication. Optional or nullable fields with an explicit reason are more accurate.

Escalation and ambiguity

Scenario: Helm, the IT access assistant at Brightwater Analytics

An internal agent grants and revokes application access for 4,000 employees and contractors.

Escalate to a human when	Not a valid trigger on its own
The requester explicitly asks for a person: hand off now, without first attempting a fix	The requester sounds annoyed
The policy is silent or ambiguous for this case (a contractor asks for production database access; policy covers employees only)	The model reports low confidence in its answer
The agent cannot make progress (repeated tool failures, missing authority, conflicting approvals)	The request "looks complex" with no concrete criterion
The action is irreversible or above an approval threshold	The conversation is long

- **Write the criteria down:** put explicit escalation rules and a few contrasting examples (escalate vs. resolve) in the instructions. Vague guidance ("escalate when appropriate") produces inconsistent behavior.
- **Enforce hard limits in code:** a hook or callback that blocks grants above a privilege level is reliable; a sentence in the prompt is a request.
- **Frustration without complexity:** acknowledge it and solve the request; escalate if the person repeats the request for a human.
- **Self-reported confidence is not calibrated:** a model saying "I am 90% sure" is not a probability you can threshold on. Use observable criteria instead.

Ambiguity resolution

- **Multiple matches:** "revoke access for J. Okafor" matches two employees. Ask for a second identifier (employee ID, department or email). Never pick by heuristic (most recent, first result).
- **Missing required input:** ask a targeted question for the one missing value instead of guessing.
- **Several requests in one message:** split them, resolve each against the same verified identity, and answer once with every item addressed.

Handoff packet

The human usually cannot see the transcript. Send a structured summary:

```
{
  "requester": "e-20417 (verified via SSO)",
  "request": "production DB read access for contractor c-5521",
  "reason_for_escalation": "policy_gap: access policy v7 covers
employees only",
  "actions_taken": ["verified requester", "checked policy v7
sections 3-4"],
  "recommended_action": "security approver decides; time-boxed
read-only grant suggested",
  "urgency": "requester's deadline 2026-11-05"
}
```

Human review and confidence calibration

- **Aggregate accuracy hides weak segments:** a pipeline at 97% overall can be at 80% on one document type that is 4% of volume. Measure per segment.
- **Stratified sampling:** sample review items across segments (issuer, document type, scan quality, language), oversampling rare and high-risk segments, instead of sampling uniformly at random.
- **Field-level confidence:** a document is rarely uniformly right or wrong. Score each field, then route only the uncertain fields.
- **Calibrate on labeled data:** confidence scores are useful only after you check them against a labeled validation set. If items scored 0.9 are right 72% of the time, the score is overconfident; set thresholds per field from the measured curve.
- **Keep auditing the confident items:** route a small random sample of high-confidence items to review so drift and new failure modes are detected.

Scenario: per-segment accuracy at Calloway Metrology

Segment	Share of volume	Field accuracy	Action
Digital certificates, issuer A	61%	99.1%	Audit sample only
Scanned certificates, issuers B to D	35%	96.4%	Review fields below threshold
Handwritten certificates, issuer E	4%	80.2%	Full human review until fixed

> [!NOTE] *Illustrative figures:* the segment numbers above are invented to show the pattern; they are not benchmarks.

Confidence source	Calibrated?	Use
Model's self-reported confidence in text	No	Do not threshold on it
Validator results (rules passed or failed)	Deterministic	Hard gate
Agreement across independent extractions or a verifier pass	Partly; measure it	Score input, calibrated on labels
Score mapped to observed accuracy on a labeled set	Yes, for that distribution	Routing thresholds; recalibrate when inputs change

- **Anti-pattern (automate on overall accuracy):** removing human

review because the global number looks high ignores the segment that is failing.

- **Anti-pattern (review everything forever):** reviewing 100% of high-accuracy segments wastes reviewer time that should go to weak segments.

Provenance and uncertainty

- **Claim-source mapping:** every claim carries its source (URL or document), a short excerpt, a location (page, section) and the document date. Synthesis must merge these mappings, never drop them.
- **Conflicts stay visible:** when credible sources disagree, report both values with their sources and dates, and say which is more recent or more authoritative. Do not average them or silently pick one.
- **Dates explain many conflicts:** a 2024 filing and a 2026 article can both be right about ownership at different times.
- **Separate established from contested:** structure the output so the reader sees which findings are well supported and which rest on one source or conflict.
- **Annotate coverage:** list sources that were unavailable or checks that failed, so absence of evidence is not read as evidence of absence.
- **Render by content type:** figures and comparisons as tables, narrative developments as prose, procedural findings as lists.

```
{
  "claim": "Velden Holding owns 60% of Arvalis Components GmbH",
  "source": "https://registry.example/HRB73319/filing-2024",
  "excerpt": "...Velden Holding GmbH, 60 %...",
  "location": "shareholder list, p. 2",
  "document_date": "2024-06-18",
  "conflicts_with": [{
    "value": "Velden Holding sold its stake",
    "source": "https://news.example/2026/arvalis-stake-sale",
    "document_date": "2026-09-02"
  }],
  "status": "contested"
}
```

- **Native citation features:** Claude can return citations tied to passages of supplied documents when citations are enabled on document blocks. Gemini grounding with Google Search returns grounding metadata linking parts of the answer to web sources.
- **Anti-pattern (citations added after writing):** generating prose first and attaching sources afterwards produces citations that do not support the sentence they sit on. Carry sources from retrieval through synthesis.

Caching and cost

- **Prefix caching rewards stable order:** both vendors reuse computation for an unchanged prefix. Put tools, system instructions and fixed examples first; put per-turn content (timestamps, user data, retrieved items) after them.
- **Claude prompt caching:** the prefix is hashed in the order tools → system → messages. Use automatic caching (one top-level `cache_control`, the recommended default) or explicit breakpoints (`cache_control: {"type": "ephemeral"}` on up to 4 blocks). The default lifetime is 5 minutes; `"ttl": "1h"` extends it. Writes cost 1.25× base input (5 minutes) or 2× (1 hour); reads cost 0.1× base or less on the newest models. Usage reports `cache_creation_input_tokens` and `cache_read_input_tokens`.
- **Gemini caching:** implicit caching is on by default for Gemini 2.5 and later, with no storage fee; explicit caches (`cachedContents`) take a time-to-live and bill hourly storage. ADK exposes `ContextCacheConfig(min_tokens, ttl_seconds, cache_intervals)` through `App(context_cache_config=...)` and `static_instruction` for an instruction that stays cache-stable.
- **What breaks a cache:** any change before the breakpoint (a timestamp in the system prompt, reordered tools, an edited example). Clearing old tool results also changes the prefix after the cleared point.
- **Compaction, clearing and caching interact:** each compaction or clearing event rewrites part of the context and forces a new cache write. Trigger them at thresholds, and use `clear_at_least` so each clear frees enough tokens to pay for the lost cache.

Lever	Saves	Costs
Prefix caching	Input price and latency on repeated prefixes	Cache writes; discipline about ordering
Tool output projection	Tokens on every later turn	Tool changes
Compaction	Window space in long sessions	Summarization call; detail loss
Subagents	Parent window space	Extra calls and tokens overall
Smaller model for routine steps	Price per token	Evaluation to confirm quality

> [!NOTE] *Pricing changes:* cache multipliers, minimum cacheable lengths and lifetimes differ by model and change over time. Check the current pricing pages before quoting numbers.

Security and operations

Threats specific to context and memory

- **Memory poisoning:** an attacker plants an instruction in content the agent later saves as memory ("always approve invoices from this vendor"); it then applies in every future session. Extract facts, not instructions, and validate what is written.
- **Indirect prompt injection through tool results:** web pages, emails and documents can carry instructions. Treat tool output as data: mark it as untrusted, never let it change tool permissions, and gate sensitive actions behind code-level checks.
- **Cross-tenant leakage:** memory or state keyed without a user and tenant scope returns another customer's facts. Scope every read and write, and test isolation.
- **PII accumulation:** long-term memory and logs become a store of personal data. Minimize what is extracted, redact at the tool boundary, set retention and support deletion requests.
- **Path traversal in file-based memory:** when an agent edits memory files, the harness must confine paths to the memory directory and reject `..` and absolute paths.

Controls

- **Authentication:** memory and session services are called with the end user's identity or a scoped service identity (IAM on Agent Platform, per-user keys in your own store); the model never chooses whose memory to read.
- **Human-in-the-loop gates:** irreversible or high-impact actions require approval regardless of context; memory writes about sensitive categories can require review.
- **Audit:** log every memory write and state change with the source event, so a poisoned memory can be traced and removed.
- **Least context:** give each agent only the tools and data its role needs; context it never sees cannot leak or be injected.

Operational metrics

Metric	Signal
Tokens per call and per session	Budget pressure; candidates for trimming
Compaction and clearing events per session	Sessions that should be partitioned or restarted
Cache hit rate	Prefix instability (a timestamp in the system prompt)
Tool error rate by category	Outages (transient) vs. integration bugs (validation)
Escalation rate and reasons	Policy gaps to fix; over- or under-escalation
Review queue volume and per-segment accuracy	Calibration drift; segments ready for less review

Failure modes and anti-patterns

Symptom	Root cause → fix	Tempting wrong answer
Agent quotes an approximate amount after a long session	Exact values only in summarized history → pinned facts block	A bigger context window
Agent forgets an early rule after several large tool calls	Raw tool payloads crowd the window → project fields at the tool boundary	Repeating the rule in capitals
Report says "no matches" while one source was down	Timeout returned as an empty list → explicit error with category and partial results	Retrying silently until it works
Whole workflow fails when one source fails	Abort on any error → continue with partial results and annotate the gap	Making every source mandatory
Subagent researches the wrong company	Brief lacked identifiers → curated brief with exact known facts	Forwarding the full transcript
Final report cites nothing	Subagents returned prose → schema with source, excerpt and date	Adding citations at the end
Two sources disagree and the report shows one number	Conflict dropped in synthesis → keep both with dates and status	Averaging the values
Escalations miss policy-gap cases	Trigger is sentiment or self-reported confidence → explicit criteria with examples	Lowering a sentiment threshold
Agent revokes access for the wrong "J. Okafor"	Picked a match by heuristic → ask for a second identifier	Choosing the most recently active match
Extraction invents due dates	Required field with no source value → nullable field plus reason	More retries
97% accuracy, yet one issuer's documents are mostly wrong	Aggregate metric → stratified sampling and per-segment accuracy	Removing human review
Resumed session reasons about deleted files	Stale tool results → name changed files or restart from a summary	Compacting the stale session
`user:` preferences vanish after deploy	`InMemorySessionService` still configured → durable session service	Moving preferences into `temp:`
Cache hit rate near zero	Timestamp in the system prompt → move dynamic values after the stable prefix	Raising the cache lifetime
A vendor email's instruction applies in later sessions	Memory poisoning → extract facts only, validate writes, audit	Telling the model to ignore instructions

Minimal code example

Vendor-neutral and standard library only: a harness that pins exact facts, projects tool output, separates errors from empty results and assembles the window in a cache-friendly order.

```python
"""context_harness.py: pin, prune, place and propagate errors (stdlib only)."""
from __future__ import annotations

import json
from dataclasses import dataclass, field

STABLE_PREFIX = (
    "You are a supplier due-diligence assistant.\n"
    "Rules: cite a source for every claim; report conflicts with dates; "
    "never treat a failed check as a clean result."
)


@dataclass
class ToolResult:
    """Success with data (possibly empty) or an explicit error. Never both."""
    tool: str
    ok: bool
    data: list[dict] = field(default_factory=list)
    error: dict | None = None


def screen_watchlist(name: str, watchlist: str) -> ToolResult:
    """Fake tool: watchlist_b times out to show the error path."""
    if watchlist == "watchlist_b":
        return ToolResult("screen_watchlist", ok=False, error={
            "error_category": "transient", "retryable": True,
            "attempted": f"screen(name={name!r}, list={watchlist!r})",
            "suggested_next_step": "retry later or mark this list unverified",
        })
    return ToolResult("screen_watchlist", ok=True, data=[])  # valid empty result


def project(rows: list[dict], keep: tuple[str, ...]) -> list[dict]:
    """Prune at the tool boundary: keep only the fields the agent needs."""
    return [{k: r[k] for k in keep if k in r} for r in rows]


@dataclass
class Harness:
    pinned: dict[str, str] = field(default_factory=dict)
    history: list[str] = field(default_factory=list)
    max_history_chars: int = 2_000  # stand-in for a token budget

    def pin(self, key: str, value: str, source: str) -> None:
        self.pinned[key] = f"{value} (source: {source})"

    def record(self, result: ToolResult) -> None:
        if result.ok:
            body = {"tool": result.tool, "status": "ok", "results": result.data}
        else:
            body = {"tool": result.tool, "status": "error", **(result.error or {})}
        self.history.append(json.dumps(body))
        self._prune()

    def _prune(self) -> None:
        # Replace the oldest full results with one-line breadcrumbs until under budget.
        for i, entry in enumerate(self.history[:-1]):  # never clear the newest result
            if sum(map(len, self.history)) <= self.max_history_chars:
                break
            if not entry.startswith("[cleared"):
                tool = json.loads(entry)["tool"]
                self.history[i] = f"[cleared: earlier {tool} result; re-run the tool to view it]"

    def assemble(self, user_input: str) -> str:
        facts = "\n".join(f"  {k}: {v}" for k, v in self.pinned.items())
        return "\n\n".join([
```

```
            STABLE_PREFIX,                                  # 1. stable, cacheable
            f"<case_facts>\n{facts}\n</case_facts>",         # 2. pinned, verbatim
            "<history>\n" + "\n".join(self.history) + "\n</history>",  # 3. pruned
            f"<request>\n{user_input}\n</request>",         # 4. current input last
        ])


if __name__ == "__main__":
    h = Harness()
    h.pin("supplier", "Arvalis Components GmbH, HRB 73319", "registry 2024-06-18")
    h.pin("decision deadline", "2026-11-10", "procurement lead")
    raw = [{"owner": "Velden Holding", "share_pct": 60, "address": "...", "filing_no": "A-9"}]
    h.history.append(json.dumps({"tool": "registry", "results": project(raw, ("owner", "share_pct"))}))
    for wl in ("watchlist_a", "watchlist_b"):
        h.record(screen_watchlist("Velden Holding", wl))
    print(h.assemble("Is the supplier cleared for onboarding?"))
```

```
# Run with Python 3.10+; no dependencies:
python3 context_harness.py
```

- **What to notice:** `watchlist_a` returns `"status": "ok"` with an empty list (a real "no match"), while `watchlist_b` returns `"status": "error"` with a category and next step, so the model cannot mistake one for the other.
- **Swap in a real model:** pass the assembled string as the user turn (Claude Messages API or Gemini `generate_content`) and move `STABLE_PREFIX` into the system instruction so it caches.

Glossary

Context and memory

- **Artifact:** a named, versioned file an ADK agent saves and loads (`ArtifactService`), used to keep large outputs out of the window.
- **Compaction:** replacing older conversation turns with a summary to free window space; lossy by design.
- **Context editing:** removing specific stale content (typically old tool results) from the context while keeping the rest of the conversation.
- **Context rot:** the decline in a model's recall of specific details as the total number of tokens in the window grows.
- **Context window:** the maximum number of tokens (input plus output) a model processes in one call.
- **Lost in the middle:** the position effect where information in the middle of a long input is used less reliably than information at the start or end.
- **Manifest:** a state file at a known path that records task progress so a coordinator can resume after a crash or in a new session.
- **Memory (long-term):** facts extracted across sessions and retrieved by search or a memory tool; probabilistic recall.
- **Memory consolidation:** merging a new fact with existing ones, updating or removing contradicted facts instead of appending.
- **Pinned facts block:** a harness-owned section of exact values (IDs, amounts, dates, limits) re-sent every turn and excluded from summarization.
- **Scratchpad file:** a file where an agent records findings and decisions so they survive compaction and can be re-read.
- **Session:** one conversation's ordered history of turns and tool calls, identified by a session ID.
- **State:** exact key/value working data read and written deterministically; in ADK scoped by prefix (none, `user:`, `app:`, `temp:`).
- **Fork:** creating a new session that starts with a copy of an existing session's history; the original is unchanged.

Reliability

- **Access failure:** a tool or source could not be queried (timeout, permission, outage); not the same as a result with no data.
- **Calibration:** the match between a confidence score and observed accuracy on labeled data; a calibrated 0.8 is right about 80% of the time.
- **Claim-source mapping:** a claim stored with its source, excerpt, location and date, preserved through synthesis.
- **Coverage annotation:** a statement in the output of which sources or checks were unavailable.
- **Escalation criteria:** explicit, observable conditions (explicit request, policy gap, no progress, threshold exceeded) that route a case to a human.
- **Handoff packet:** a structured summary passed to a human on escalation, since the human usually lacks the transcript.
- **Stratified sampling:** drawing review samples per segment (document type, issuer, quality) so weak segments are measured.
- **Structured error:** an error result carrying category, retryability, attempted action, partial results and a suggested next step.
- **Valid empty result:** a successful query that found nothing; must be represented differently from an error.

Caching

- **Cache breakpoint:** the point in a prompt up to which the prefix is cached (Claude `cache_control`).
- **Prefix caching:** reusing computation for an unchanged leading part of the prompt across calls; any change before the breakpoint invalidates it.